



Xi'an Jiaotong-Liverpool University

西交利物浦大學

SCHOOL OF ADVANCED TECHNOLOGY

SAT301 FINAL YEAR PROJECT

*Waveform Optimization for Multicarrier Modulation:
A Deep Reinforcement Learning Approach for NOMA
Scheduling*

Final Thesis

In Partial Fulfillment
of the Requirements for the Degree of
Bachelor of Science

Student Name	:	Zichen Qiu
Student ID	:	2252705
Supervisor	:	Chunchuan Lyu
Assessor	:	Ye Liu

Abstract

In multicarrier non-orthogonal multiple access (NOMA) systems, the way users are scheduled across subcarriers directly shapes the transmitted waveform and determines the overall transmission latency. This scheduling problem can be formulated as a combinatorial optimization task of assigning jobs to machines with the objective of minimizing the makespan, where each machine may process up to two jobs simultaneously under mutual interference. Since the problem is NP-hard, we propose a deep reinforcement learning (DRL) approach to learn efficient scheduling policies directly from the problem structure. Specifically, we adopt a customized policy-gradient-based reinforcement learning algorithm, where the policy network is built upon graph neural networks and attention mechanisms, enabling the agent to handle varying problem sizes. The model is trained on instances with 20 jobs and 5 machines and tested on larger configurations to assess generalization. Experimental results show that the trained agent achieves performance roughly equivalent to the widely-used LPT heuristic but does not yet surpass it, and its generalization ability remains limited. This work demonstrates the potential of DRL for waveform optimization via NOMA scheduling, while also highlighting the need for further improvements in scalability and solution quality. The code is available at [GitHub Code](#).

Keywords: Non-orthogonal multiple access (NOMA); user scheduling; Reinforcement learning; combinatorial optimization; makespan minimization

Acknowledgements

First and foremost, I would like to express my sincere gratitude to my supervisor, Dr. Chunchuan Lyu, who has been a constant source of guidance and inspiration throughout my undergraduate research. Having known him for over three years, I have benefited immensely from his teaching in machine learning, artificial intelligence, and reinforcement learning. He patiently answered my countless questions and led me from simple explorations in prompt engineering to more substantial research in machine learning and reinforcement learning. His mentorship has been invaluable to my academic growth.

I am deeply thankful to my parents for their unconditional love and unwavering support. They not only respected my decision to pursue a BSc in Information and Computing Science but also provided the necessary financial and emotional backing that allowed me to focus on my studies. My interest in computers, which began in secondary school, was able to flourish because of their belief in me.

My sincere thanks also go to the many friends who have accompanied me during these four years. They have offered me comfort and strength through life's ups and downs, shared exam revision strategies, collaborated with me on group assignments, and discussed academic questions large and small. Their companionship made the undergraduate journey far less lonely and much more joyful.

I am grateful to all the teachers and staff at XJTLU for their dedication and for providing me with a solid foundation in computer science—courses such as computer networks and computer organization have prepared me well for future challenges. The research activities I participated in, together with the machine learning and reinforcement learning knowledge I acquired outside the classroom, have laid a firm groundwork for my graduate studies.

Looking ahead, I am eager to continue exploring the field of artificial intelligence in my upcoming postgraduate journey. This dissertation is not only a conclusion to my undergraduate life, but also a starting point for deeper inquiry.

Contents

Abstract	ii
Acknowledgements	iii
List of Figures	vi
List of Acronyms	vii
1 Introduction	1
2 Background	4
2.1 Longest Processing Time (LPT) Scheduling Baseline	4
2.2 Reinforcement Learning Fundamentals	7
3 Problem Formulation	11
3.1 System Model and Problem Formulation	11
3.2 Markov Decision Process Formulation	13
3.3 Simulation Parameters	14
4 Methodology	16
4.1 Policy Network Architecture	16
4.1.1 Encoder: Job and Machine Feature Embedding	16
4.1.2 Graph Neural Network with Edge-Aware Attention	17
4.1.3 Policy Head and Action Selection	18
4.1.4 Policy Wrapper and Decoding Loop	19
4.2 Training Algorithm	19
4.2.1 Policy Gradient Objective	20
4.2.2 Value Function Learning	21
4.2.3 Optimisation Procedure	21
5 Experiments	23
5.1 Experimental Setup	23
5.1.1 Data Generation	23
5.1.2 Early-Stage Experiments	24
5.1.3 Main Experiments	24
5.2 Results	24
5.2.1 Early-Stage Exploration	24
5.2.2 Main Experiments	26

5.3	Discussion	28
6	Conclusion and Future Work	29
6.1	Conclusion	29
6.2	Future Work	30
	References	32

List of Figures

1.1	Illustration of NOMA pairing and makespan calculation.	2
3.1	Illustration of the assignment constraints for $M = 2$ machines and $K = 4$ jobs. Two matrices $\mathbf{I}^1$ and $\mathbf{I}^2$ collect the indicators I_{jk}^1 and I_{jk}^2 . The lower triangular part is irrelevant. The dashed rectangles mark the elements that must sum to 1, ensuring each job is scheduled exactly once.	13
5.1	Training curves of the 4-40 model evaluated on different job ranges. Makespan is normalised by the random baseline.	25
5.2	Training curves of the 60-80 model evaluated on different job ranges. . . .	25
5.3	Comparison of average makespan for different models and test sizes. . . .	26
5.4	Comparison of average makespan on test sets with $K \in \{20, 30, 40, 50, 60\}$	27
5.5	Makespan of four independently trained 20-job models.	27

List of Acronyms

Term	Initial components of the term
DRL	Deep Reinforcement Learning
GAE	Generalized Advantage Estimation
LPT	Longest Processing Time
MDP	Markov Decision Process
MLP	Multi-Layer Perceptron
NOMA	Non-Orthogonal Multiple Access
OMA	Orthogonal Multiple Access
PPO	Proximal Policy Optimization
RL	Reinforcement Learning
SIC	Successive Interference Cancellation
TD	Temporal-Difference

Chapter 1 Introduction

The soaring demand for mobile data and the proliferation of delay-sensitive applications are pushing orthogonal multiple access (OMA) schemes to their fundamental limits. In OMA, each user is assigned exclusive time-frequency resources; while simple to implement, this orthogonality wastes spectrum when resources are allocated to users with poor channel conditions [1]. Non-orthogonal multiple access (NOMA) has emerged as a key enabler for future wireless networks because it allows multiple users to share the same resource block simultaneously [2]. By employing successive interference cancellation (SIC) at the receiver, NOMA reuses the bandwidth of weak-channel users for strong-channel users, thereby significantly boosting spectral efficiency and supporting more concurrent transmissions [3].

Beyond throughput enhancement, NOMA also offers the potential to reduce communication latency – a critical requirement for applications such as autonomous driving, industrial control, and real-time video [4, 5]. In this context, the problem of scheduling uplink transmissions to minimize the maximum completion time (makespan) has attracted considerable attention. The scheduling problem can be described as follows: given a set of devices with different packet sizes and channel gains, the base station must assign each device to a sub-carrier and decide whether devices are served alone or paired in a NOMA fashion, with the goal of finishing all transmissions as early as possible. When two devices are paired, mutual interference extends the processing time of the stronger device, creating a trade-off that makes the optimization highly non-trivial.

Figure 1.1 gives a conceptual illustration of how NOMA pairing and the resulting makespan are derived. A complete mathematical formulation of the scheduling problem is deferred to Chapter 3.

The makespan minimization problem in NOMA-enabled systems has been proven NP-hard [6]. Exact algorithms become computationally prohibitive for realistic network sizes, motivating the use of heuristic and learning-based approaches. Traditional heuristics, such as the Longest Processing Time (LPT) rule or simulated annealing, can produce acceptable solutions but are often hand-crafted and struggle to generalize across varying network conditions [7, 8]. Recently, deep reinforcement learning (DRL) has shown remarkable success in solving complex combinatorial optimization problems [9] by learning policies directly from

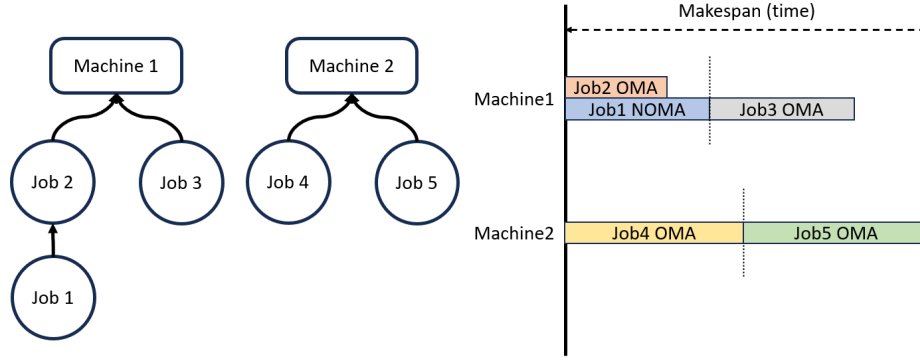


Figure 1.1 Illustration of NOMA pairing and makespan calculation.

data, offering the potential to discover scheduling strategies that surpass human-designed heuristics [10].

The principal aim of this work is to investigate whether a DRL agent can learn effective NOMA scheduling policies and to critically evaluate its performance against a well-known heuristic baseline. Specifically, our objectives are:

1. To formulate the NOMA makespan minimization problem in a manner amenable to DRL, including the design of a suitable state representation, action space, and reward signal.
2. To implement a customized policy-gradient-based reinforcement learning algorithm whose policy network incorporates graph neural networks and attention mechanisms, enabling the agent to handle problem instances of varying dimensions.
3. To train the agent on synthetic datasets of a fixed size and test its generalization ability on larger instances not seen during training.
4. To compare the obtained policies with the LPT algorithm, identify performance gaps, and discuss the limitations of the current learning-based approach.

By tackling the NOMA scheduling problem through the lens of DRL, this project contributes to the growing body of research on learning-aided resource allocation and highlights the challenges that remain before such methods can be practically deployed.

The remainder of this dissertation is organized as follows. Chapter 2 provides background on the LPT algorithm and reinforcement learning. Chapter 3 formulates the NOMA scheduling problem mathematically and specifies the experimental parameters. Chapter 4 details our MDP modeling and the customized reinforcement learning algorithm. Chapter 5 presents the

experimental setup, results, and analysis. Finally, Chapter 6 concludes the work and outlines future directions.

Chapter 2 Background

To contextualise the solution techniques explored in this dissertation, the present chapter introduces two foundational pillars that underpin the subsequent methodology and evaluation. The first is the Longest Processing Time (LPT) heuristic—a classical dispatching rule for makespan minimisation—which serves both as a performance benchmark and as a source of intuition for scheduling decisions [7, 8]. The second is reinforcement learning (RL), a paradigm that enables an agent to learn decision-making policies through interaction with an environment [11]. Together, these two topics provide the necessary background for understanding how a learning-based scheduler can be constructed and why it is compared against a well-established heuristic. The chapter begins with a description of the LPT algorithm and its extension to NOMA settings, and then proceeds to outline the core concepts of RL, including policy gradient methods [12].

2.1 Longest Processing Time (LPT) Scheduling Baseline

Since the considered NOMA scheduling problem is NP-hard [6], obtaining globally optimal solutions becomes computationally infeasible for large-scale instances [13]. Consequently, practical studies commonly rely on heuristic scheduling algorithms that offer a favourable trade-off between computational complexity and solution quality. In this work, we adopt a two-stage NOMA-aware Longest Processing Time (LPT) algorithm as the primary benchmark for evaluating the proposed deep reinforcement learning approach.

Graph-Based NOMA Pairing

Consider the set of jobs

$$\mathcal{K} = \{1, 2, \dots, K\}. \quad (2.1)$$

For any two jobs j and k satisfying $j < k$, the corresponding NOMA transmission time T_{jk} is defined according to the system model in Chapter 3. Each feasible NOMA pair can therefore be interpreted as an edge in an undirected weighted graph

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}), \quad (2.2)$$

where each vertex in $\mathcal{V}$ represents one job and each edge $e_{jk} \in \mathcal{E}$ corresponds to pairing jobs j and k .

To construct favourable NOMA pairs, we assign the edge weight

$$w_{jk} = \Delta - T_{jk}, \quad (2.3)$$

where Δ is a sufficiently large constant satisfying

$$\Delta > \max_{j,k} T_{jk}. \quad (2.4)$$

Under this definition, smaller pair completion times correspond to larger edge weights. Therefore, maximizing the total matching weight is equivalent to minimizing the aggregate transmission time of the selected NOMA pairs.

The pairing problem can then be formulated as a maximum weight matching problem:

$$\max_{\mathcal{M} \subseteq \mathcal{E}} \sum_{e_{jk} \in \mathcal{M}} w_{jk}, \quad (2.5)$$

subject to the constraint that each vertex participates in at most one matched edge. Classical matching algorithms, such as Weighted Maximum Matching in General Graphs by Zvi Galil, can solve this problem in polynomial time with computational complexity $O(K^3)$ [14, 15].

After the matching stage, the original scheduling problem is transformed into a set of composite jobs. Each composite job corresponds either to:

- a NOMA pair (j, k) with processing time T_{jk} , or
- an unpaired stand-alone job k with processing time T_{kk} .

LPT-Based Subcarrier Scheduling

Once the composite jobs are obtained, the remaining task reduces to a classical identical-machines scheduling problem. Let

$$\mathcal{T} = \{T_1, T_2, \dots, T_N\} \quad (2.6)$$

denote the set of processing times associated with all composite jobs after the pairing stage.

The LPT scheduling rule first sorts all composite jobs in non-increasing order:

$$T_{(1)} \geq T_{(2)} \geq \cdots \geq T_{(N)}. \quad (2.7)$$

The jobs are then assigned sequentially to the subcarrier currently having the smallest accumulated workload.

More formally, let L_m denote the current load of subcarrier m . At each iteration, the next composite job $T_{(i)}$ is assigned according to

$$m^* = \arg \min_{m \in \{1, \dots, M\}} L_m, \quad (2.8)$$

after which the corresponding load is updated as

$$L_{m^*} \leftarrow L_{m^*} + T_{(i)}. \quad (2.9)$$

The final makespan produced by the heuristic algorithm is therefore

$$C_{\max}^{\text{LPT}} = \max_{m \in \{1, \dots, M\}} L_m. \quad (2.10)$$

For the classical identical-machines scheduling problem, the LPT algorithm satisfies the approximation bound [16]

$$\frac{C_{\max}^{\text{LPT}}}{C_{\max}^*} \leq \frac{4}{3} - \frac{1}{3M}, \quad (2.11)$$

where $C_{\max}^*$ denotes the optimal makespan. Although this bound does not directly extend to the NOMA-aware setting because of the additional pairing stage, the combined matching-LPT framework remains widely adopted due to its low computational cost and consistently strong empirical performance.

The overall complexity of the two-stage scheduling framework is dominated by the maximum weight matching procedure, resulting in total complexity

$$O(K^3), \quad (2.12)$$

while the subsequent LPT scheduling stage contributes only linear-time assignment overhead after sorting.

In this dissertation, the above matching-LPT algorithm serves as the principal heuristic

baseline. Experimental results in Chapter 5 compare the proposed reinforcement learning policy against this handcrafted scheduling strategy in terms of makespan minimization and generalization ability.

2.2 Reinforcement Learning Fundamentals

Reinforcement learning (RL) provides a general framework for solving sequential decision-making problems through interaction with an environment [11]. Unlike supervised learning, where labelled input-output pairs are provided explicitly, RL agents improve their behaviour by maximizing long-term cumulative rewards obtained from trial-and-error exploration [17, 18]. Owing to its ability to handle high-dimensional state spaces and combinatorial action structures, RL has recently attracted considerable attention in communication networks, resource allocation, and scheduling optimization problems [19].

Markov Decision Process Formulation

A reinforcement learning problem is commonly modeled as a Markov decision process (MDP) [20], represented by the tuple

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma), \quad (2.13)$$

where:

- $\mathcal{S}$ denotes the state space;
- $\mathcal{A}$ denotes the action space;
- $\mathcal{P}(s'|s, a)$ denotes the transition probability from state s to state s' under action a ;
- $\mathcal{R}(s, a)$ denotes the immediate reward function (expected reward);
- $\gamma \in [0, 1)$ is the discount factor.

At each decision step t , the agent observes the current state $S_t \in \mathcal{S}$, selects an action $A_t \in \mathcal{A}$, receives a reward R_{t+1} , and transitions to the next state S_{t+1} . The Markov property assumes that future state transitions depend only on the current state-action pair:

$$\Pr(S_{t+1}|S_t, A_t, S_{t-1}, \dots, S_0) = \Pr(S_{t+1}|S_t, A_t). \quad (2.14)$$

The agent behaviour is governed by a policy

$$\pi(a|s) = \Pr(A_t = a \mid S_t = s), \quad (2.15)$$

which specifies a probability distribution over feasible actions. The objective of RL is to determine an optimal policy π^* maximizing the expected cumulative discounted return

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}. \quad (2.16)$$

The expected long-term utility of a policy is characterized by the state-value function

$$V^\pi(s) = \mathbb{E}_\pi [G_t \mid S_t = s], \quad (2.17)$$

and the action-value function

$$Q^\pi(s, a) = \mathbb{E}_\pi [G_t \mid S_t = s, A_t = a]. \quad (2.18)$$

These functions satisfy the Bellman expectation equations [20]:

$$V^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left[\mathcal{R}(s, a) + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}(s'|s, a) V^\pi(s') \right], \quad (2.19)$$

$$Q^\pi(s, a) = \mathcal{R}(s, a) + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}(s'|s, a) \sum_{a' \in \mathcal{A}} \pi(a'|s') Q^\pi(s', a'). \quad (2.20)$$

The optimal policy is obtained by maximizing the value functions:

$$V^*(s) = \max_{\pi} V^\pi(s), \quad Q^*(s, a) = \max_{\pi} Q^\pi(s, a). \quad (2.21)$$

The optimal value functions satisfy the Bellman optimality equations

$$V^*(s) = \max_{a \in \mathcal{A}} \left[\mathcal{R}(s, a) + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}(s'|s, a) V^*(s') \right], \quad (2.22)$$

$$Q^*(s, a) = \mathcal{R}(s, a) + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}(s'|s, a) \max_{a' \in \mathcal{A}} Q^*(s', a'). \quad (2.23)$$

When the state and action spaces are finite and small, these equations can be solved by dynamic programming methods such as value iteration or policy iteration. In combinatorial optimization problems, however, the state and action spaces often grow exponentially with

the problem dimension, rendering exact dynamic programming approaches computationally impractical.

Policy Gradient Optimization [12]

In policy-based reinforcement learning, the agent's policy is a parameterized function, typically a neural network with parameters θ , denoted by π_θ . Given a state s_t , the policy outputs a distribution over actions, from which an action a_t is sampled. A trajectory $\tau = \{s_1, a_1, s_2, a_2, \dots, s_T, a_T\}$ represents a complete episode. The probability of a trajectory under π_θ is

$$p_\theta(\tau) = p(s_1) \prod_{t=1}^T p_\theta(a_t \mid s_t) p(s_{t+1} \mid s_t, a_t). \quad (2.24)$$

The total reward of a trajectory is $R(\tau)$, and the expected reward is

$$\bar{R}_\theta = \sum_{\tau} R(\tau) p_\theta(\tau) = \mathbb{E}_{\tau \sim p_\theta(\tau)} [R(\tau)]. \quad (2.25)$$

To maximize $\bar{R}_\theta$, we compute its gradient with respect to θ :

$$\nabla \bar{R}_\theta = \sum_{\tau} R(\tau) \nabla p_\theta(\tau). \quad (2.26)$$

Using the identity

$$\nabla f(x) = f(x) \nabla \log f(x), \quad (2.27)$$

we obtain

$$\nabla p_\theta(\tau) = p_\theta(\tau) \nabla \log p_\theta(\tau). \quad (2.28)$$

Hence,

$$\begin{aligned} \nabla \bar{R}_\theta &= \sum_{\tau} R(\tau) p_\theta(\tau) \nabla \log p_\theta(\tau) \\ &= \mathbb{E}_{\tau \sim p_\theta(\tau)} [R(\tau) \nabla \log p_\theta(\tau)]. \end{aligned} \quad (2.29)$$

The log-probability of a trajectory expands as

$$\log p_\theta(\tau) = \log p(s_1) + \sum_{t=1}^T \log p_\theta(a_t \mid s_t) + \sum_{t=1}^T \log p(s_{t+1} \mid s_t, a_t). \quad (2.30)$$

Since $p(s_1)$ and $p(s_{t+1} \mid s_t, a_t)$ come from the environment and do not depend on θ , their gradients vanish, leaving

$$\nabla \log p_\theta(\tau) = \sum_{t=1}^T \nabla \log p_\theta(a_t \mid s_t). \quad (2.31)$$

Substituting this into (2.29) gives the full policy gradient expression:

$$\begin{aligned} \nabla \bar{R}_\theta &= \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[R(\tau) \sum_{t=1}^T \nabla \log p_\theta(a_t \mid s_t) \right] \\ &\approx \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} R(\tau^n) \nabla \log p_\theta(a_t^n \mid s_t^n), \end{aligned} \quad (2.32)$$

where the expectation is approximated by N sampled trajectories $\{\tau^n\}$. The parameters are updated via gradient ascent:

$$\theta \leftarrow \theta + \eta \nabla \bar{R}_\theta, \quad (2.33)$$

with learning rate η . In practice, trajectories are collected by running the current policy π_θ in the environment, recording state-action pairs and the resulting return $R(\tau^n)$. The final gradient estimator is compactly written as

$$\nabla \bar{R}_\theta = \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} R(\tau^n) \nabla \log p_\theta(a_t^n \mid s_t^n). \quad (2.34)$$

Equation (2.34) shows that actions in trajectories with higher total reward $R(\tau^n)$ are reinforced by increasing their log-probability, while actions with lower reward are suppressed.

Chapter 3 Problem Formulation

This chapter provides a rigorous mathematical description of the NOMA-aware scheduling problem and frames it as a Markov decision process suitable for deep reinforcement learning. Section 3.1 introduces the system model and formulates the makespan minimisation problem as a constrained combinatorial optimization. Section 3.2 recasts the problem as a sequential decision-making task, defining state space, action space, rewards, and the mechanics of constructing a solution step by step. Finally, Section 3.3 specifies the simulation parameters used throughout the experimental study.

3.1 System Model and Problem Formulation

Consider a system with K jobs and M identical machines. Each job represents an uplink transmission from a device, and each machine corresponds to a subcarrier. A machine can process at most two jobs simultaneously; when it processes a single job, we refer to that job as being served in *stand-alone* mode. A job must be assigned to exactly one machine, and once a machine starts working on a job, it continues until the job is completed.

For stand-alone processing, the transmission speed of job k is given by the Shannon-Hartley formula

$$R_k^{\text{OMA}} = W \cdot \log_2 \left(1 + \frac{h_k P}{n} \right), \quad k = 1, 2, \dots, K, \quad (3.1)$$

where W is the subcarrier bandwidth, P the fixed transmit power, n the additive white Gaussian noise power, and h_k the channel gain of device k . Without loss of generality, we order the jobs so that

$$h_1 \geq h_2 \geq \dots \geq h_K.$$

If two jobs j and k with $h_j \geq h_k$ are paired and processed together on the same machine, the stronger job j suffers from interference caused by job k . Its speed becomes

$$R_j^{\text{NOMA}} = W \cdot \log_2 \left(1 + \frac{h_j P}{h_k P + n} \right), \quad (3.2)$$

while the weaker job k retains its stand-alone speed R_k^{OMA} . The time required to complete

the pair is therefore the larger of the two individual completion times:

$$T_{jk} = \begin{cases} \frac{L_k}{R_k^{\text{OMA}}}, & j = k, \\ \max\left\{\frac{L_j}{R_j^{\text{NOMA}}}, \frac{L_k}{R_k^{\text{OMA}}}\right\}, & j < k, \end{cases} \quad (3.3)$$

where L_k denotes the amount of data (in bits) to be transmitted by job k .

The scheduling decisions are encoded by binary variables $I_{jk}^m \in \{0, 1\}$ for $m = 1, \dots, M$ and $1 \leq j \leq k \leq K$. Specifically,

- $I_{jj}^m = 1$ if job j is assigned to machine m in stand-alone mode;
- $I_{jk}^m = 1$ (with $j < k$) if jobs j and k are paired together on machine m .

Every job must be scheduled exactly once, which is enforced by the two sets of constraints below.

With these definitions, the makespan (maximum completion time among all machines) is expressed as

$$C_{\max} = \max_{m=1, \dots, M} \sum_{j=1}^K \sum_{k=j}^K I_{jk}^m \cdot T_{jk}. \quad (3.4)$$

The optimisation problem is

$$\underset{I_{jk}^m}{\text{minimise}} \quad \max_{m=1, \dots, M} \sum_{j=1}^K \sum_{k=j}^K I_{jk}^m \cdot T_{jk}, \quad (3.5)$$

$$\text{subject to} \quad I_{jk}^m \in \{0, 1\}, \quad m = 1, \dots, M, \quad j = 1, \dots, K, \quad k \geq j, \quad (3.6)$$

$$\sum_{m=1}^M \sum_{k=j}^K I_{1k}^m = 1, \quad j = 2, \dots, M, \quad (3.7)$$

$$\sum_{m=1}^M \sum_{k=j}^K I_{jk}^m + \sum_{m=1}^M \sum_{k=1}^{j-1} I_{kj}^m = 1, \quad j = 2, \dots, M. \quad (3.8)$$

Constraints (3.7) and (3.8) guarantee that each job appears in exactly one scheduling entry. Figure 3.1 illustrates the structure of these constraints for a small example.

The problem is known to be NP-hard [6], which motivates the use of learning-based heuristics.

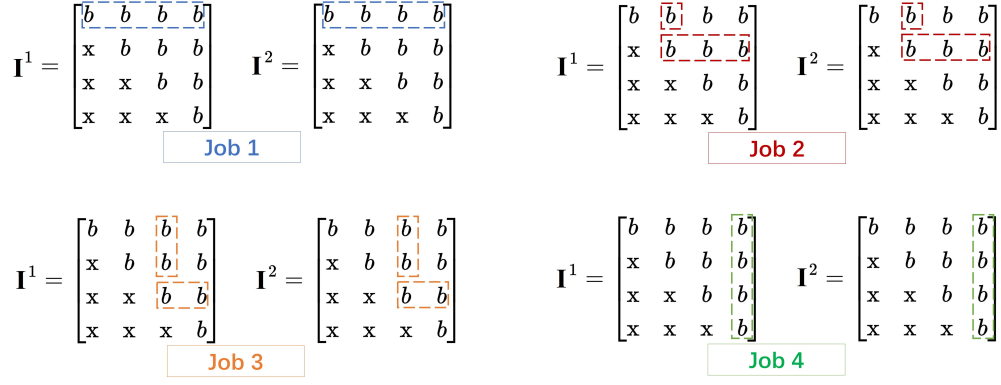


Figure 3.1 Illustration of the assignment constraints for $M = 2$ machines and $K = 4$ jobs. Two matrices $\mathbf{I}^1$ and $\mathbf{I}^2$ collect the indicators I_{jk}^1 and I_{jk}^2 . The lower triangular part is irrelevant. The dashed rectangles mark the elements that must sum to 1, ensuring each job is scheduled exactly once.

3.2 Markov Decision Process Formulation

To apply deep reinforcement learning, the scheduling problem is cast as a finite-horizon Markov decision process (MDP) [20]. The agent constructs a complete schedule sequentially, adding one assignment edge at each step.

State Space

We represent a partially built schedule by a binary matrix $G \in \{0, 1\}^{K \times (K+M)}$. An entry $G_{ij} = 1$ has the following meaning:

- If $j \leq K$, job i is paired with job j (forming a NOMA pair).
- If $j > K$, job i is assigned to machine $j - K$ (stand-alone or as part of a pair with another job attached to that machine).

A valid schedule must respect the following constraints at every step:

- **Each job appears exactly once:** $\forall i, \sum_j G_{ij} = 1$.
- **Each job can be paired at most once:** $\forall j \leq K, \sum_i G_{ij} \leq 1$.
- **No self-loops or cycles:** $G_{ij}G_{ji} = 0$ for all i, j .
- **Lower triangular part is zero:** $G_{ij} = 0$ whenever $i \leq j$ (to avoid redundancy).

Additionally, the matrix G must correspond to a feasible pairing structure (no job is paired with more than one other job). The state includes the current matrix G as well as any aux-

iliary information needed by the neural network, such as node features (device vs. machine indicator, channel gain h_k , data length L_k , and the pairwise completion times T_{jk}).

Action Space

At each decision step, the agent selects a valid position (i, j) to set $G_{ij} = 1$. The set of legal actions is determined by a mask that hides entries that would violate any of the above constraints. The action space is thus discrete, with size scaling quadratically in K .

State Transition

The transition is deterministic: after placing $G_{ij} = 1$, the matrix is updated accordingly, and the mask is recomputed. The episode terminates when all jobs have been assigned (i.e., when the row-sum constraint is satisfied for every row), yielding a complete schedule.

Reward

Since the objective is to minimise makespan, we define the final reward as

$$R_T = -C_{\max},$$

where $C_{\max}$ is computed from the final schedule via (3.4). Intermediate steps receive zero reward (sparse reward setting) unless reward shaping is applied (discussed later in Chapter 4).

Policy and Value Networks

Because the state and action spaces are large and structured, we employ a policy network $\pi_{\theta}(a|s)$ that outputs a probability distribution over legal actions and a value network $V_{\phi}(s)$ that estimates the expected return. Their architectures, based on graph neural networks and attention mechanisms, are detailed in Chapter 4.

3.3 Simulation Parameters

The following physical-layer parameters, adopted from [6] and used in our later experiments, define the environment from which training and testing instances are drawn.

1. **Transmit power:** $P = 20$ dBm (0.1 W).

2. **Noise power:** The noise power spectral density is -174 dBm/Hz. The total bandwidth is fixed at 180 kHz, so the per-subcarrier bandwidth is $W = 180/M$ kHz. Consequently, the noise power is

$$n = \frac{10^{-17.4}}{1000} \times W \times 10^3 = 10^{-20.4} W.$$

3. **Channel gains:** The path loss between device k and the receiver is modelled as

$$\text{PL}(d_k) = 128.1 + 37.6 \log_{10}(d_k) \text{ dB},$$

where d_k (in km) is drawn uniformly from $[0.1, 0.5]$. The linear gain is then

$$h_k = 10^{-\text{PL}(d_k)/10}.$$

4. **Job lengths:** Each L_k is an integer uniformly distributed between 1 and 1024 bits.
5. **System sizes:** Training is performed on instances with $K = 20$ jobs and $M = 5$ machines. For evaluation, we test on (K, M) pairs where $K \in \{20, 30, 40, 50, 60\}$ and $M = 5$.

All logarithms in rate formulas use base 2, so speeds are measured in bits per second. The chosen range of distances ensures a maximum path loss difference of approximately 26 dB, which provides a realistic variety of channel conditions.

Chapter 4 Methodology

This chapter details the neural network architecture and the training algorithm developed to solve the NOMA scheduling problem formulated in Chapter 3. Section 4.1 describes the policy network, which combines a job-machine feature encoder, a graph neural network with edge-aware attention, and an autoregressive decoding scheme. Section 4.2 presents the modified policy gradient algorithm used to train the policy and value networks. The algorithm builds upon the proximal policy optimization (PPO) framework [21], but replaces the standard advantage estimation and value loss with components tailored to the single-step Markov decision process (MDP) [20] view of constructive scheduling.

Together with the MDP formulation established in Chapter 3, this chapter directly addresses Objectives 1 and 2 stated in the Introduction. Objective 1 (formulating the problem in a DRL-compatible manner) is realized by the state space, action space, and reward design of Section 3.2, while the customized policy-gradient algorithm with graph neural networks and attention mechanisms detailed in the present chapter fulfills Objective 2.

4.1 Policy Network Architecture

The policy is parameterised by a neural network $\pi_{\theta}(a|s)$ that takes as input the current state s (represented by the partially constructed matrix G , node features, and problem parameters) and outputs a probability distribution over all feasible actions. The architecture consists of three main modules: an encoder that projects raw features into a latent space, a graph neural network that performs relational reasoning over the partial schedule, and a decoder that computes action logits. The complete model is implemented as a subclass of `nn.Module` [22] and interfaces with the environment through the `GNNPolicy` wrapper described in Section 4.1.4.

4.1.1 Encoder: Job and Machine Feature Embedding

For each instance, we construct a node for every job and every machine, resulting in $K + M$ nodes. Each node is associated with a 10-dimensional feature vector. For job i ($1 \leq i \leq K$), the features are:

- binary indicator for job type (0 for job, 1 for machine);

- a one-hot encoding indicating that the node is a job;
- normalised channel gain h_i and data length L_i ;
- the job index i (padded with zeros for dummy positions when $i > \text{actual job count}$);
- the remaining entries are filled with zeros for the machine-related features.

For machine m ($1 \leq m \leq M$), the features include:

- the machine indicator set to 1;
- a one-hot encoding for the machine type;
- the per-subcarrier bandwidth W , transmit power P , and noise power N ;
- the machine index m (zero-padded).

All scalar features are normalised to similar numerical ranges.

The raw feature tensor $\mathbf{F} \in \mathbb{R}^{B \times (K+M) \times 10}$ is fed into a multi-layer perceptron (MLP) [23] composed of a single Transformer encoder layer [24, 22] (with 5 attention heads and feed-forward dimension $4d_{\text{model}}$) followed by a linear projection and a LeakyReLU activation [25]. Let d_{model} denote the embedding dimension. The Transformer encoder processes the entire set of nodes in parallel, capturing global interactions, and the output is linearly mapped to the embedding space:

$$\mathbf{H}^{(0)} = \text{LeakyReLU}(\text{TransformerEncoder}(\mathbf{F}) \mathbf{W}_{\text{proj}} + \mathbf{b}_{\text{proj}}) \in \mathbb{R}^{B \times (K+M) \times d_{\text{model}}}. \quad (4.1)$$

The initial embeddings $\mathbf{H}^{(0)}$ are then refined by a residual linear layer and layer normalisation, yielding the encoder output $\mathbf{H}^{\text{enc}}$ that serves as the memory for the subsequent graph-based processing.

4.1.2 Graph Neural Network with Edge-Aware Attention

The partially built schedule is represented by the binary matrix $G \in \{0, 1\}^{K \times (K+M)}$ (see Section 3.2). To enable relational reasoning over the existing assignments, we construct a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ where each node corresponds to a job or a machine. An edge $j \rightarrow i$ exists if job i has been assigned to target j (another job or a machine), and the edge

is annotated with the scalar T_{ij} taken from the precomputed time matrix. This edge feature captures the processing time associated with the already scheduled pair.

The graph neural network employs the EdgeGATConv layer [26] proposed in SCENE [27]. Let $\mathbf{v}_i$ be the feature vector of node i and $\mathbf{e}_{j,i}$ the scalar edge feature for the directed edge $j \rightarrow i$. The node update rule at each convolutional layer is

$$\mathbf{v}'_i = \Theta_s \mathbf{v}_i + \sum_{j \in \mathcal{N}(i)} \alpha_{j,i} (\Theta_n \mathbf{v}_j + \Theta_e \mathbf{e}_{j,i}), \quad (4.2)$$

where Θ_s is a learnable weight matrix for the self-transformation, Θ_n for the neighbour features, and Θ_e for the edge features. The attention coefficient $\alpha_{j,i}$ is computed as

$$\alpha_{j,i} = \text{softmax}_i \left(\text{LeakyReLU} \left(\mathbf{a}^T [\Theta_n \mathbf{v}_i \parallel \Theta_n \mathbf{v}_j \parallel \Theta_e \mathbf{e}_{j,i}] \right) \right), \quad (4.3)$$

with $\mathbf{a}$ being a learnable weight vector and $\parallel$ denoting concatenation. The softmax normalization is taken over all incoming edges to node i . Multi-head attention is employed by concatenating N_{heads} independently parameterised transformations; in our implementation, $N_{\text{heads}} = 8$.

In our architecture, we stack two EdgeGATConv layers. After each convolution, layer normalization is applied per head. The output of the last graph convolutional layer is averaged over the heads and then passed through a Transformer decoder layer, which uses the encoder outputs $\mathbf{H}^{\text{enc}}$ as memory to further refine the node representations. The resulting tensor $\mathbf{H}^{\text{dec}}$ captures both local schedule structure and global problem context.

4.1.3 Policy Head and Action Selection

From the decoder output $\mathbf{H}^{\text{dec}} \in \mathbb{R}^{B \times (K+M) \times d_{\text{model}}}$, we extract two components: a query matrix $\mathbf{Q} = \mathbf{H}^{\text{dec}} \mathbf{W}_q \in \mathbb{R}^{B \times (K+M) \times d_k}$ and a key matrix $\mathbf{K} = \mathbf{H}_{1:K}^{\text{dec}} \in \mathbb{R}^{B \times K \times d_k}$ (only the job portion). The unnormalised compatibility scores between any job and any possible target are computed as

$$\text{score}_{ij} = (\mathbf{Q}_i \cdot \mathbf{K}_j), \quad i = 1, \dots, K+M, j = 1, \dots, K. \quad (4.4)$$

These scores are reshaped into a vector of length $K(K+M)$, corresponding to the flattened action space. A softmax is applied over the legal actions (those not masked by the environ-

ment) to obtain a probability distribution:

$$\pi_{\theta}(a|s) = \frac{\exp(\text{score}_a)}{\sum_{a' \in \mathcal{A}_{\text{legal}}(s)} \exp(\text{score}_{a'})}, \quad (4.5)$$

where the mask sets the score of illegal actions to $-\infty$. To improve numerical stability and encourage exploration, we further apply a tanh scaling to the logits:

$$\log \pi_{\theta}(a|s) = \tau \cdot \tanh\left(\frac{\log \text{softmax}(\text{score}_a)}{\tau}\right), \quad (4.6)$$

with scaling factor τ (set to 10 in our experiments). This logit transformation is commonly used in combinatorial optimisation policies [28] to avoid extreme probabilities early in training.

During training, actions are sampled from the categorical distribution; during validation and testing, we select the action with the highest probability (greedy decoding).

4.1.4 Policy Wrapper and Decoding Loop

The top-level module `GNNPolicy` manages the interaction with the environment. For a given batch of initial states, it repeatedly calls the encoder and decoder to obtain logits, masks illegal actions, samples or selects an action according to the decoding strategy, updates the state via the environment’s `step` function, and records the log probabilities of the chosen actions. The process terminates when all jobs have been assigned. The module returns the sequence of actions, the sum of log probabilities, and (optionally) the entropy of the policy distribution. The log-likelihood of a trajectory is given by

$$\log \pi_{\theta}(\tau|s_0) = \sum_{t=0}^{T-1} \log \pi_{\theta}(a_t|s_t), \quad (4.7)$$

where T is the number of steps until the schedule is complete.

4.2 Training Algorithm

We adopt a policy gradient method built upon the proximal policy optimization (PPO) framework [21], but tailored to the constructive scheduling MDP [20] described in Section 3.2. In standard PPO, an episode consists of multiple state transitions and Generalized Advantage Estimation (GAE) [29] is typically employed. In our setting, the entire schedule is constructed

autoregressively, and a single reward is returned at the terminal step. Thus, the whole trajectory is treated as a one-shot decision under the policy, and the advantage reduces to the difference between the received return and a learned baseline. We further replace the conventional Huber value loss [30] with a quantile regression formulation for improved robustness.

4.2.1 Policy Gradient Objective

Let $\tau = (s_0, a_0, s_1, a_1, \dots, s_{T-1}, a_{T-1}, s_T)$ be a complete schedule generated by the policy π_θ , where $T = K$ (one assignment per job). The terminal state s_T contains a full assignment matrix, from which the makespan $C_{\max}$ is computed. The raw environmental reward is defined through a logarithmic mapping that emphasises relative improvements in makespan:

$$\tilde{R}(\tau) = -\ln(\alpha C_{\max}(\tau) + \epsilon), \quad (4.8)$$

with $\alpha = 500$ and $\epsilon = 10^{-3}$. This transformation compresses the scale of large makespans and stabilises learning. During training, the rewards are further standardised over the batch to have zero mean and unit variance, yielding the normalised return $R(\tau)$.

The critic network $V_\phi(s_0)$ estimates the expected normalised return from the initial state s_0 (the empty graph). For a batch of N trajectories $\{\tau^n\}_{n=1}^N$, the policy gradient can be estimated using the REINFORCE [31] with baseline paradigm. Because the policy’s log-probability factorises over the decoding steps,

$$\log \pi_\theta(\tau|s_0) = \sum_{t=0}^{T_n-1} \log p_\theta(a_t^n | s_t^n),$$

the gradient of the expected return with respect to the policy parameters is approximated by

$$\nabla_\theta \bar{R}_\theta \approx \frac{1}{N} \sum_{n=1}^N (R(\tau^n) - V_\phi(s_0^n)) \sum_{t=1}^{T_n} \nabla_\theta \log p_\theta(a_t^n | s_t^n). \quad (4.9)$$

In practice, we replace the naive gradient estimator with the clipped surrogate objective of PPO [21] to avoid destructively large policy updates. Let $r^n(\theta) = \exp(\log \pi_\theta(\tau^n | s_0^n) - \log \pi_{\theta_{\text{old}}}(\tau^n | s_0^n))$ be the probability ratio with respect to the policy that collected the trajectory. The advantage for trajectory n is

$$A^n = R(\tau^n) - V_\phi(s_0^n). \quad (4.10)$$

The policy is optimised by maximising

$$\mathcal{L}^{\text{policy}}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{n=1}^N \min\left(r^n(\boldsymbol{\theta}) A^n, \text{clip}(r^n(\boldsymbol{\theta}), 1 - \epsilon, 1 + \epsilon) A^n\right), \quad (4.11)$$

where the clipping parameter ϵ is set to 0.2.

4.2.2 Value Function Learning

The critic is trained to predict the normalised return by minimising a loss function. A conventional choice is the Huber loss, whose gradient with respect to the critic parameters ϕ is

$$\nabla_{\phi} \hat{\mathcal{L}} \approx \nabla_{\phi} \frac{1}{N} \sum_{n=1}^N \text{HuberLoss}(V_{\phi}(s_0^n), R(\tau^n)). \quad (4.12)$$

The Huber loss provides a smooth trade-off between mean squared error and mean absolute error, making it less sensitive to outliers [30].

In this work, however, the distribution of makespans is often skewed, with occasional very large values that can dominate the gradient signal. To obtain a more robust value estimate, we employ a quantile regression loss that focuses on a pre-specified quantile of the error distribution. Define the prediction error $\delta^n = V_{\phi}(s_0^n) - R(\tau^n)$. The quantile loss at quantile q (we choose $q = 0.7$) is

$$\mathcal{L}^{\text{value}}(\phi) = \frac{1}{N} \sum_{n=1}^N w(\delta^n) (\delta^n)^2, \quad w(\delta) = \begin{cases} q, & \delta > 0, \\ 1 - q, & \delta \leq 0. \end{cases} \quad (4.13)$$

When $q > 0.5$, positive errors (underestimation of the return) are penalised more heavily, which encourages conservative value predictions. This conservatism is advantageous in risk-sensitive scheduling because it biases the policy towards safer (i.e., faster) schedules. The quantile loss replaces the Huber loss in our final algorithm.

4.2.3 Optimisation Procedure

Algorithm 1 summarises the training loop. We use a fixed batch size of 128 problem instances per iteration. After resetting the environment, the current policy generates a complete schedule for each instance and records the log-probabilities. The critic evaluates the

initial states, and the normalised rewards are computed. The collected trajectories form the on-policy buffer.

We perform $K_{\text{epochs}} = 2$ PPO epochs over the buffer. In each epoch, the buffer is divided into mini-batches of size 32. For each mini-batch, the policy and critic are re-evaluated, and the surrogate loss (4.11) and quantile value loss (4.13) are computed. The policy and critic are updated using separate Adam optimisers [32] with an initial learning rate of 10^{-4} . Gradient norms of the policy network are clipped to a maximum of 0.5; no gradient clipping is applied to the critic. The total training objective is the sum of the two losses, with the entropy bonus coefficient set to zero in our experiments. After training, the policy is evaluated on unseen problem sizes to assess its generalisation ability.

Algorithm 1 Training Procedure (Modified PPO)

```

1: for iteration = 1, 2, ... do
2:   Sample a batch of  $B = 128$  instances
3:   Generate schedules using  $\pi_{\theta}$ ; store  $\log p_{\text{old}}$ 
4:   Compute normalised rewards  $R$  from makespans
5:   Evaluate critic  $V_{\phi}(s_0)$ 
6:   Fill buffer  $\mathcal{D} = \{(\log p_{\text{old}}, R, \dots)\}$ 
7:   for epoch = 1, ...,  $K_{\text{epochs}}$  do
8:     for each mini-batch  $\mathcal{M} \subset \mathcal{D}$  do
9:       Re-evaluate  $\log \pi_{\theta}$  and  $V_{\phi}$ 
10:      Ratio  $r = \exp(\log \pi_{\theta} - \log p_{\text{old}})$ 
11:      Advantage  $A = R - V_{\phi}$ 
12:      Surrogate loss  $\mathcal{L}^{\text{policy}}$  via (4.11)
13:      Quantile loss  $\mathcal{L}^{\text{value}}$  via (4.13)
14:      Update  $\theta$  with gradient clipping; update  $\phi$ 
15:     end for
16:   end for
17: end for

```

The algorithm above captures the key modifications over standard PPO [21]: (i) single-step advantage without GAE [29], (ii) quantile loss for value estimation, and (iii) independent optimisation of policy and critic with separate gradient clipping. These adaptations are driven by the constructive nature of the scheduling MDP [20] and the desire for robust value predictions in the face of highly variable makespans.

Chapter 5 Experiments

This chapter evaluates the proposed deep reinforcement learning scheduler against several baseline methods. We first describe the experimental setup, including the training configurations and the data generation procedure. Then we present results from two phases of experiments: an early exploration phase with varying problem sizes and a final evaluation phase that matches the simulation settings introduced in Chapter 3. Finally, we discuss the observed performance trends and analyse the reasons behind the strengths and limitations of the learned policies.

In doing so, this chapter directly addresses Objectives 3 and 4 stated in the Introduction. Objective 3 (training on fixed-size data and testing generalization to larger instances) is realized by our experimental design, while the comparison with the LPT heuristic and the discussion of performance gaps fulfill Objective 4.

5.1 Experimental Setup

All models are implemented in PyTorch [22] and trained using the modified PPO algorithm described in Chapter 4. The policy and value networks are optimised with separate Adam optimisers with an initial learning rate of 10^{-4} . The batch size is set to 128 problem instances, and the PPO clipping parameter ϵ is 0.2. Training runs for a fixed number of steps that depends on the problem size; details are given below for each phase. The entropy bonus coefficient is set to zero, and gradient clipping with a maximum norm of 0.5 is applied only to the policy network.

5.1.1 Data Generation

In every experiment, problem instances are generated on the fly using the procedure of Section 3.3. For a given pair (K, M) , the channel gains h_k and data lengths L_k are sampled from the distributions specified there. The total bandwidth is fixed at 180 kHz and equally divided among the M machines, yielding a per-subcarrier bandwidth of $W = 180/M$ kHz. Other physical-layer parameters, such as the transmit power $P = 0.1$ W and the noise power spectral density of -174 dBm/Hz, are kept constant.

5.1.2 Early-Stage Experiments

In the initial exploration, we trained two separate models on instances with very different job ranges in order to study generalisation:

- A **4-40 model**, trained on instances where K is uniformly sampled between 4 and 40, with $M = 5$.
- A **60-80 model**, trained on instances with $K \in [60, 80]$ and $M = 5$.

During this phase, the simulation parameters were slightly different from the final version (e.g., a different value of W and an older power allocation scheme), so the absolute makespan values are not directly comparable to later results. The main purpose of this phase was to assess whether the policy can generalise across input sizes and to compare against a simple random scheduling baseline.

5.1.3 Main Experiments

For the final evaluation, we align the simulation setup exactly with that of the reference paper [6]. We focus on a single configuration: training with $K = 20$ jobs and $M = 5$ machines, which captures the core NOMA pairing and scheduling complexity while allowing fast iteration. The trained policy is then tested on five different problem sizes: $K \in \{20, 30, 40, 50, 60\}$, always with $M = 5$. This tests the model's ability to generalise to larger unseen instances.

5.2 Results

5.2.1 Early-Stage Exploration

Figure 5.1 shows the performance of the 4-40 model during training, evaluated periodically on five held-out test sets with $K = 4-20, 20-40, 40-60, 60-80$ and $80-90$ jobs. The vertical axis reports the average makespan, normalised by the performance of a random scheduling policy. The model quickly outperforms the random baseline on the smaller test ranges (4-20 and 20-40). However, on the larger ranges (60-80 and 80-90), the makespan fluctuates heavily and remains unstable even in later stages of training, indicating limited generalisation to input sizes far from the training distribution.



Figure 5.1 Training curves of the 4-40 model evaluated on different job ranges. Makespan is normalised by the random baseline.

Figure 5.2 repeats the same evaluation for the 60-80 model. In the early stage of training, the performance on the 80-90 test set is unstable, but after convergence the model achieves lower makespans than the 4-40 model on *all five* test ranges, including the small-job settings that it never encountered during training. This demonstrates a better ability to generalise across input sizes.

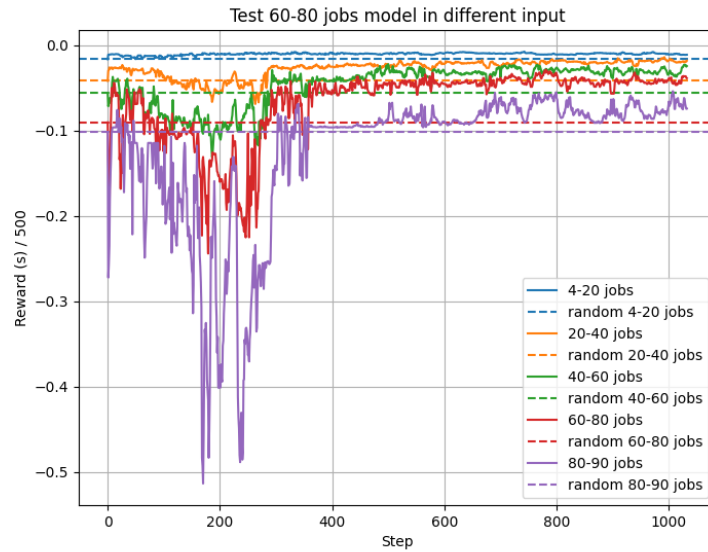


Figure 5.2 Training curves of the 60-80 model evaluated on different job ranges.

A summary comparison is provided in Figure 5.3. The bar chart displays the average

makespan of the random baseline, the 4-40 model and the 60-80 model on three representative test sets (small, medium, large). The 60-80 model consistently achieves the shortest makespan; on average it saves up to 45.8% of the time compared with the random baseline.

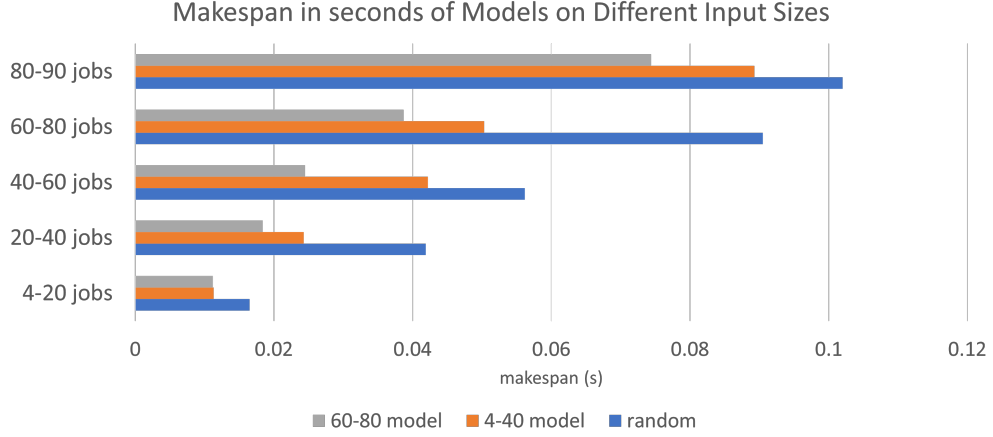


Figure 5.3 Comparison of average makespan for different models and test sizes.

The early-stage experiments yield three key observations:

1. The 60-80 model can save as much as 45.8% time on average compared with the random baseline.
2. Both models generalise to input sizes that were not seen during training.
3. Training on larger instances (the 60-80 model) leads to a more robust and better-performing policy overall.

5.2.2 Main Experiments

In the main evaluation, we compare the 20-job-trained DRL policy against four baselines:

- **NOMA_LPT**: the two-stage LPT heuristic described in Section 2.1.
- **OMA_LPT**: an LPT scheduler that forbids any NOMA pairing (only stand-alone jobs).
- **NOMA_random**: a random pairing followed by random machine assignment.
- **OMA_random**: a purely random scheduler with no pairing.

Figure 5.4 presents the average makespan as a function of the number of jobs K for all five methods. On the training size ($K = 20$), the DRL policy performs roughly on par with

NOMA_LPT. As K increases, the gap between the DRL policy and NOMA_LPT widens; for $K = 60$, the DRL policy even becomes worse than the random NOMA baseline. This trend indicates that the policy fails to generalise to larger instances despite being trained on a fixed size.

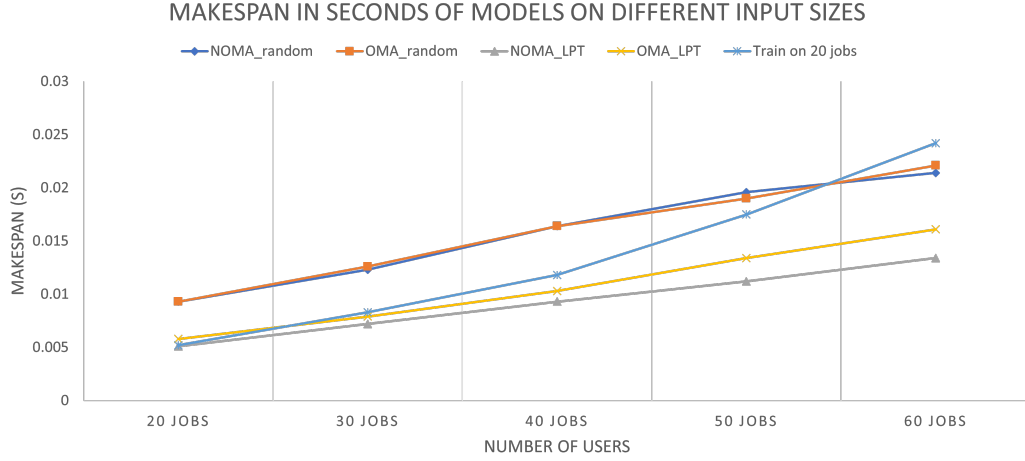


Figure 5.4 Comparison of average makespan on test sets with $K \in \{20, 30, 40, 50, 60\}$.

We also conducted several training runs with different random seeds and hyper-parameter adjustments. Figure 5.5 shows the makespan curves of four top-performing models on the 20-job test set. While the seeds produce slightly different convergence speeds, the final makespans are very close to each other and to the LPT baseline. None of the variations significantly surpass LPT.

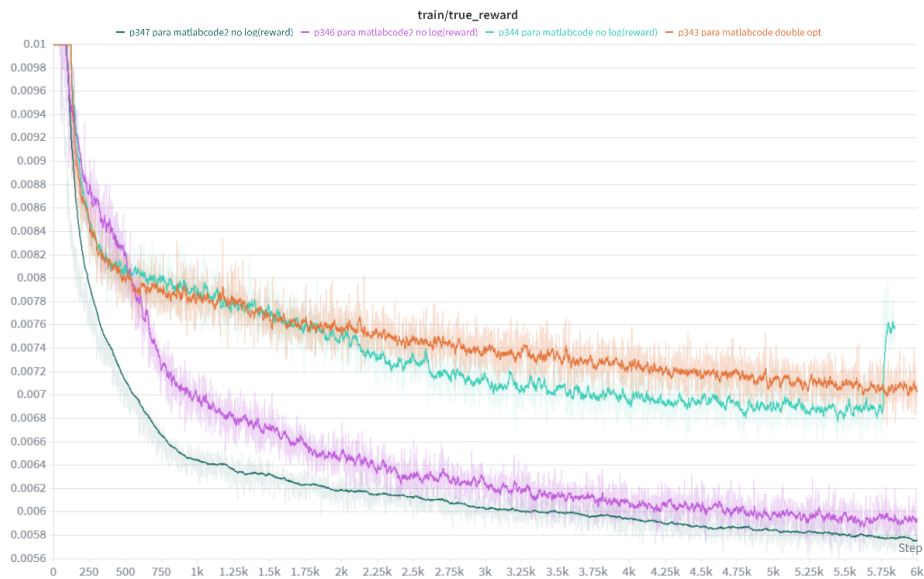


Figure 5.5 Makespan of four independently trained 20-job models.

The main experimental results can be summarised as follows:

- The DRL model performs roughly the same as NOMA_LPT on the training size ($K = 20$).
- The model does not outperform the LPT heuristic on any tested size.
- Generalisation to larger instances is poor; at $K = 60$ the makespan is even higher than that of the random NOMA scheduler.

5.3 Discussion

The performance patterns observed in the two phases offer several insights. First, the early-stage experiments confirm that the DRL policy is capable of learning meaningful scheduling strategies and can generalise to some extent across different input sizes, especially when trained on a wide range of instances. The superior performance of the 60-80 model over the 4-40 model suggests that exposing the policy to larger and more diverse scheduling configurations during training helps the network learn more robust features. This is likely because larger instances contain more varied NOMA pairing patterns and workload distributions.

However, the main experiments reveal clear limitations. When trained on a single size ($K = 20$), the policy fails to transfer its knowledge to larger instances. Several factors may contribute to this shortcoming. The first is the limited capacity of the current policy network. Although the graph neural network captures local pairwise relationships, it may not adequately propagate the global workload information that is crucial for makespan minimisation when the instance size grows. The second factor is the training data diversity: a fixed problem size does not expose the agent to the combinatorial structures that appear in larger systems. The third factor is the difficulty of credit assignment in the constructive MDP; the sparse reward signal provides little guidance on intermediate decisions, making it hard for the policy to learn long-term planning skills that would generalise.

To summarise, the DRL approach demonstrates viability for NOMA scheduling but does not yet offer a clear advantage over well-tuned heuristics. The results point towards several directions for future improvement, including training on a mixture of instance sizes, designing richer reward signals (e.g., reward shaping based on intermediate makespan estimates), and exploring more powerful model architectures.

Chapter 6 Conclusion and Future Work

6.1 Conclusion

This dissertation investigated the application of deep reinforcement learning to the NP-hard problem of makespan minimisation in uplink multi-carrier NOMA systems. The scheduling task was formulated as a combinatorial optimisation problem over binary assignment variables and was cast into a Markov decision process suitable for policy gradient methods. A customised policy network integrating graph neural networks with edge-aware attention was designed to handle the variable-sized bipartite scheduling graph, and a modified PPO algorithm with quantile value loss was used for training.

Experimental results obtained in two phases confirm that DRL is a viable approach for NOMA scheduling. In early-stage studies, the policy generalised across a broad range of problem sizes and reduced the makespan by up to 45.8% compared with a random baseline, demonstrating that the network can extract meaningful structural patterns. However, the main evaluation, conducted under a fixed training size of 20 jobs and 5 machines, revealed that the learned policy performs only on par with the well-established LPT heuristic and fails to generalise to larger instances. The performance degradation for unseen job counts suggests that the current policy network and training paradigm do not yet capture the combinatorial complexity required for robust scaling.

A central challenge lies in the sparse reward structure inherent to constructive scheduling: only a single scalar makespan is available at the end of the episode, providing limited supervision for intermediate decisions. This credit assignment difficulty, combined with a fixed training distribution, restricts the agent’s ability to develop long-term planning skills that transfer across problem scales. Despite these limitations, the work confirms that model-free DRL can serve as a building block for learning NOMA resource allocation policies, and it identifies clear directions for improvement.

With respect to the four objectives set out in Chapter 1:

1. **Objective 1 (Formulation):** The NOMA scheduling problem was successfully formulated as an MDP with a compact state representation, a variable-sized action space, and a sparse terminal reward (Chapter 3). This formulation proved suitable for policy

gradient learning.

2. **Objective 2 (Algorithm):** A customized policy-gradient algorithm was implemented, featuring a graph neural network with edge-aware attention and a PPO-based training procedure (Chapter 4). The architecture handles varying problem sizes and complex NOMA pairing interactions.
3. **Objective 3 (Generalization):** The agent was trained on fixed-size instances and tested on larger unseen configurations. While early-stage experiments showed moderate generalization, the main evaluation indicated that the policy trained on 20 jobs does not scale well to 60 jobs (Chapter 5).
4. **Objective 4 (Comparison):** The learned policy was compared against the LPT heuristic. It achieved similar performance on the training size but did not surpass LPT; the performance gap widened with larger instances, highlighting limitations in the current approach (Chapter 5).

6.2 Future Work

Several avenues are promising for overcoming the current shortcomings and enhancing the performance of learning-based NOMA schedulers.

Supervised pre-training and reinforcement fine-tuning. A natural extension is to combine the strengths of imitation learning and reinforcement learning. As suggested in the neural combinatorial optimisation literature [28, 33], one can first construct a training set of high-quality solutions using the LPT heuristic (or an optimal solver for small instances) and perform supervised pre-training of the policy network. The pre-trained model can then be fine-tuned with the modified PPO algorithm. This two-stage approach provides a favourable initial policy and can significantly reduce the exploration burden during RL, often leading to more robust final solutions.

Architectural improvements. The current graph neural network relies on two Edge-GATConv layers [26]. Incorporating deeper or more expressive graph architectures, such as Graph Transformers or higher-order message passing schemes, may enable the model to capture long-range workload balance relationships more effectively [34, 35]. Additionally, attention mechanisms that explicitly model machine-level state and job-machine interactions

could improve generalisation [36].

Training on mixed instance sizes. Training exclusively on a fixed problem size limits the policy’s exposure to diverse combinatorial structures [37]. A curriculum learning strategy that gradually increases the number of jobs and machines, or simply training on a uniform mixture of sizes, could force the network to learn size-invariant scheduling principles [38]. Data augmentation techniques that exploit the permutation symmetries of jobs and machines are also worth investigating [39].

Reward shaping and auxiliary objectives. To alleviate sparse reward issues, one can design intermediate rewards based on workload balance or remaining makespan estimates [40]. Auxiliary prediction tasks, such as forecasting the final makespan from partial schedules, could also be added to the value network to provide richer training signals.

Systematic comparison with OMA and other NOMA heuristics. Finally, a rigorous comparison of the DRL scheduler against its OMA counterpart (i.e., a DRL agent restricted to stand-alone assignments) would help quantify the specific gain attributable to NOMA pairing. Similarly, benchmarking against a wider range of heuristics, such as simulated annealing and genetic algorithms, would position the learning-based method within the broader landscape of combinatorial optimisation solvers.

References

- [1] S. M. R. Islam, N. Avazov, O. A. Dobre, and K.-s. Kwak, “Power-domain non-orthogonal multiple access (noma) in 5g systems: Potentials and challenges,” *IEEE Communications Surveys & Tutorials*, vol. 19, no. 2, pp. 721–742, 2017.
- [2] Z. Ding, Y. Liu, J. Choi, Q. Sun, M. Elkashlan, C.-L. I, and H. V. Poor, “Application of non-orthogonal multiple access in lte and 5g networks,” 2016. [Online]. Available: <https://arxiv.org/abs/1511.08610>
- [3] H. Tabassum, M. S. Ali, E. Hossain, M. J. Hossain, and D. I. Kim, “Uplink vs. downlink noma in cellular networks: Challenges and research directions,” in *2017 IEEE 85th Vehicular Technology Conference (VTC Spring)*, 2017, pp. 1–7.
- [4] Z. Ding, P. Fan, and H. V. Poor, “Impact of user pairing on 5g nonorthogonal multiple-access downlink transmissions,” *IEEE Transactions on Vehicular Technology*, vol. 65, no. 8, pp. 6010–6023, 2016.
- [5] G. Jang, C. Lee, N. Kim, W. Kang, T.-V. Nguyen, and S. Cho, “Throughput maximization for uplink iot noma systems with efficient resource allocation,” in *2020 International Conference on Information and Communication Technology Convergence (ICTC)*, 2020, pp. 1763–1765.
- [6] Y. Guo, C. W. Sung, Y. Liu, and C. S. Chen, “Maximum completion time optimization in uplink multi-subcarrier noma systems,” *IEEE Wireless Communications Letters*, vol. 12, no. 7, pp. 1214–1218, 2023.
- [7] B. Yang, Y. Liu, and C. S. Chen, “Uplink scheduling in a noma-enabled single-cell wireless network using simulated annealing,” in *2023 21st International Symposium on Modeling and Optimization in Mobile, Ad Hoc, and Wireless Networks (WiOpt)*, 2023, pp. 485–492.
- [8] R. Conway, W. Maxwell, and L. Miller, *Theory of Scheduling*, ser. Dover Books on Computer Science Series. Dover, 2003. [Online]. Available: https://books.google.com.sg/books?id=Yr5_kQDa_ssC

-
- [9] K. Li, T. Zhang, R. Wang, Y. Wang, Y. Han, and L. Wang, “Deep reinforcement learning for combinatorial optimization: Covering salesman problems,” *IEEE Transactions on Cybernetics*, vol. 52, no. 12, p. 13142–13155, Dec. 2022. [Online]. Available: <http://dx.doi.org/10.1109/TCYB.2021.3103811>
- [10] T. Zhang, A. Banitalebi-Dehkordi, and Y. Zhang, “Deep reinforcement learning for exact combinatorial optimization: Learning to branch,” 2022. [Online]. Available: <https://arxiv.org/abs/2206.06965>
- [11] R. Sutton and A. Barto, “Reinforcement learning: An introduction,” *IEEE Transactions on Neural Networks*, vol. 9, no. 5, pp. 1054–1054, 1998.
- [12] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, “Policy gradient methods for reinforcement learning with function approximation,” in *Advances in Neural Information Processing Systems*, S. Solla, T. Leen, and K. Müller, Eds., vol. 12. MIT Press, 1999. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/1999/file/464d828b85b0bed98e80ade0a5c43b0f-Paper.pdf
- [13] A. Wang, L. Lei, E. Lagunas, S. Chatzinotas, and B. Ottersten, “Completion time minimization in noma systems: Learning for combinatorial optimization,” *IEEE Networking Letters*, vol. 3, no. 1, pp. 15–18, 2021.
- [14] Z. Galil, “Efficient algorithms for finding maximum matching in graphs,” *ACM Comput. Surv.*, vol. 18, no. 1, p. 23–38, Mar. 1986. [Online]. Available: <https://doi.org/10.1145/6462.6502>
- [15] H. N. Gabow, “Implementation of algorithms for maximum matching on nonbipartite graphs,” 1973. [Online]. Available: <https://api.semanticscholar.org/CorpusID:124328923>
- [16] V. T’kindt, “J.y.-t. leung, handbook of scheduling: Algorithms, models and performance analysis, computer and information science series,” *Eur. J. Oper. Res.*, vol. 170, pp. 1018–1022, 2006. [Online]. Available: <https://api.semanticscholar.org/CorpusID:207697977>

- [17] A. K. Shakya, G. Pillai, and S. Chakrabarty, “Reinforcement learning algorithms: A brief survey,” *Expert Systems with Applications*, vol. 231, p. 120495, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0957417423009971>
- [18] M. Ghasemi, A. H. Moosavi, and D. Ebrahimi, “A comprehensive survey of reinforcement learning: From algorithms to practical challenges,” 2025. [Online]. Available: <https://arxiv.org/abs/2411.18892>
- [19] Y. Gu, Z. Liu, S. Dai, C. Liu, Y. Wang, S. Wang, G. Theodoropoulos, and L. Cheng, “Deep reinforcement learning for job scheduling and resource management in cloud computing: An algorithm-level review,” 2025. [Online]. Available: <https://arxiv.org/abs/2501.01007>
- [20] R. Bellman, “A markovian decision process,” *Indiana University Mathematics Journal*, vol. 6, pp. 679–684, 1957. [Online]. Available: <https://api.semanticscholar.org/CorpusID:123329493>
- [21] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” 2017. [Online]. Available: <https://arxiv.org/abs/1707.06347>
- [22] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “Pytorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, Eds., vol. 32. Curran Associates, Inc., 2019. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf
- [23] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986. [Online]. Available: <https://api.semanticscholar.org/CorpusID:205001834>
- [24] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” 2023. [Online]. Available: <https://arxiv.org/abs/1706.03762>

- [25] A. L. Maas, “Rectifier nonlinearities improve neural network acoustic models,” 2013. [Online]. Available: <https://api.semanticscholar.org/CorpusID:16489696>
- [26] M. Wang, D. Zheng, Z. Ye, Q. Gan, M. Li, X. Song, J. Zhou, C. Ma, L. Yu, Y. Gai, T. Xiao, T. He, G. Karypis, J. Li, and Z. Zhang, “Deep graph library: A graph-centric, highly-performant package for graph neural networks.” *arXiv: Learning*, 2019. [Online]. Available: <https://api.semanticscholar.org/CorpusID:221304724>
- [27] T. Monninger, J. Schmidt, J. Rupprecht, D. Raba, J. Jordan, D. Frank, S. Staab, and K. Dietmayer, “Scene: Reasoning about traffic scenes using heterogeneous graph neural networks,” *IEEE Robotics and Automation Letters*, vol. 8, no. 3, p. 1531–1538, Mar. 2023. [Online]. Available: <http://dx.doi.org/10.1109/LRA.2023.3234771>
- [28] I. Bello, H. Pham, Q. V. Le, M. Norouzi, and S. Bengio, “Neural combinatorial optimization with reinforcement learning,” 2017. [Online]. Available: <https://arxiv.org/abs/1611.09940>
- [29] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” *CoRR*, vol. abs/1506.02438, 2015. [Online]. Available: <https://api.semanticscholar.org/CorpusID:3075448>
- [30] P. J. Huber, “Robust Estimation of a Location Parameter,” *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73 – 101, 1964. [Online]. Available: <https://doi.org/10.1214/aoms/1177703732>
- [31] R. J. Williams, “Simple statistical gradient-following algorithms for connectionist reinforcement learning,” *Machine Learning*, vol. 8, pp. 229–256, 2004. [Online]. Available: <https://api.semanticscholar.org/CorpusID:2332513>
- [32] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *CoRR*, vol. abs/1412.6980, 2014. [Online]. Available: <https://api.semanticscholar.org/CorpusID:6628106>
- [33] X. Chen and Y. Tian, “Learning to perform local rewriting for combinatorial optimization,” 2019. [Online]. Available: <https://arxiv.org/abs/1810.00337>

- [34] A. Arroyo, F. Barbero, H. Blayney, M. M. Bronstein, X. Dong, P. Lio, R. Pascanu, and P. Vandergheynst, “A survey on over-smoothing and over-squashing: Unified propagation perspectives on graph neural networks and transformers,” *Transactions on Machine Learning Research*, 2026. [Online]. Available: <https://openreview.net/forum?id=H9zhC5pVnH>
- [35] M. Besta, F. Scheidl, L. Gianinazzi, G. Kwasniewski, S. Klaiman, J. Müller, and T. Hoefler, “Demystifying higher-order graph neural networks,” 2025. [Online]. Available: <https://arxiv.org/abs/2406.12841>
- [36] B. Altundas, S. Chen, S. Singh, S. Deo, M. Cho, and M. C. Gombolay, “Heterogeneous graph transformers for simultaneous mobile multi-robot task allocation and scheduling under temporal constraints,” in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. [Online]. Available: <https://openreview.net/forum?id=k1fbdnwjCH>
- [37] C. Zhou, X. Lin, Z. Wang, X. Tong, M. Yuan, and Q. Zhang, “Instance-conditioned adaptation for large-scale generalization of neural routing solver,” *IEEE Transactions on Intelligent Transportation Systems*, p. 1–15, 2026. [Online]. Available: <http://dx.doi.org/10.1109/TITS.2026.3674538>
- [38] C. Waubert De Puiseau, H. Tercan, and T. Meisen, “Curriculum learning in job shop scheduling using reinforcement learning,” 2023. [Online]. Available: <https://www.repo.uni-hannover.de/handle/123456789/13532>
- [39] H. Nguyen, T. Kozuno, C. C. Beltran-Hernandez, and M. Hamaya, “Symmetry-aware reinforcement learning for robotic assembly under partial observability with a soft wrist,” 2024. [Online]. Available: <https://arxiv.org/abs/2402.18002>
- [40] L. Zhang, Y. Yan, C. Yang, and Y. Hu, “Dynamic flexible job-shop scheduling by multi-agent reinforcement learning with reward-shaping,” *Advanced Engineering Informatics*, vol. 62, p. 102872, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1474034624005202>